

The "Pure Python" Django Personal Website Guide

This guide outlines the industry-standard approach to building a monolithic Django website. This architecture keeps your stack simple (Python/HTML/CSS) while remaining robust enough for professional deployment.

Part 1: The Master Checklist (Order of Operations)

Follow these checkpoints in order. Do not jump ahead.

Phase 1: The Foundation (Setup)

- [] **Environment:** Create a project folder, set up a virtual environment (`venv`), and activate it.
- [] **Dependencies:** Install `django` , `python-dotenv` (for secrets), and `Pillow` (for images).
- [] **Project Start:** Run `django-admin startproject config .` (Note the dot at the end; this prevents nested folders).
- [] **Version Control:** Initialize `git` , create a `.gitignore` (Python/Django specific), and make your first commit.

Phase 2: Configuration & Architecture

- [] **Settings Refactor:** Modify `config/settings.py` to use `python-dotenv` . Never hardcode API keys or Secret Keys.
- [] **App Generation:** Create your apps based on functionality. Standard apps for a personal site:
 - `core` : Homepage, About, Contact, global base templates.
 - `portfolio` : Your projects, case studies.
 - `blog` : Articles, tutorials (optional).
- [] **Registration:** Add these new apps to `INSTALLED_APPS` in `settings.py` .

Phase 3: Database & Models

- [] **Portfolio Models:** Create `Project` and `Tag` models in `portfolio/models.py` .
- [] **Blog Models:** Create `Post` model in `blog/models.py` .
- [] **Migrations:** Run `makemigrations` and `migrate` .
- [] **Admin Panel:** Register models in `admin.py` so you can add content immediately. **Stop here and add 3 dummy projects via the admin panel.**

Phase 4: The "Python" Frontend (Templates)

- [] **Base Template:** Create `templates/base.html` . This contains your `<head>` , Navbar, and Footer. All other pages will "extend" this.

- [] **Static Files:** Set up `static/` folder for global CSS/Images. Configure `STATICFILES_DIRS` .
- [] **CSS Framework:** Connect a CSS framework (Tailwind via CDN or Bootstrap) in `base.html` to make styling easier.

Phase 5: Views & Routing

- [] **URLs:** Set up `portfolio/urls.py` and `blog/urls.py` , then include them in the main `config/urls.py` .
- [] **Views:** Write Class-Based Views (CBVs) like `ListView` and `DetailView` in your apps. They write less code than function views.
- [] **Template Connection:** Create the HTML files for the views (e.g., `project_list.html`).

Phase 6: Production Readiness

- [] **Environment Check:** Ensure `DEBUG=False` works via environment variables.
- [] **Static Serving:** Install `whitenoise` to serve static files in production.
- [] **Dependency Freeze:** Run `pip freeze > requirements.txt` .

Part 2: Industry Standard Directory Structure

This structure separates configuration (`config`) from business logic (`apps`).

```

my_personal_site/
├── .env                    # Secrets (NEVER commit this)
├── .gitignore
├── db.sqlite3
├── manage.py
├── requirements.txt
├──
├── config/                # Project container
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py        # Global settings
│   ├── urls.py            # Main URL router
│   └── wsgi.py            # Server entry point
├──
├── core/                  # "General" app
│   ├── migrations/
│   ├── templates/
│   │   └── core/          # Namespace your templates!
│   │       ├── home.html
│   │       └── about.html
│   ├── admin.py
│   ├── models.py
│   ├── views.py
│   └── urls.py
├──
├── portfolio/             # "Projects" app
│   ├── templates/
│   │   └── portfolio/
│   │       └── project_list.html
├──
└──

```

```

|   |   └─ project_detail.html
|   └─ models.py
|   └─ views.py
|
└─ static/                # CSS, JS, Images
    └─ css/
    └─ img/
|
└─ media/                 # User uploaded content (Project screenshots)
|
└─ templates/             # Global Templates
    └─ base.html          # The skeleton for every page

```

Part 3: Critical Code Implementation

Here is the code for the most critical files to ensure you start with "Production Quality" standards.

1. config/settings.py (Snippet)

Using `pathlib` and `environ` is the modern standard.

```

from pathlib import Path
import os
from dotenv import load_dotenv

# Load environment variables from .env file
load_dotenv()

BASE_DIR = Path(__file__).resolve().parent.parent

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = os.getenv('SECRET_KEY', 'insecure-default-key-for-dev')

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = os.getenv('DEBUG', 'False') == 'True'

ALLOWED_HOSTS = os.getenv('ALLOWED_HOSTS', '127.0.0.1,localhost').split(',')

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Third party
    'whitenoise.runserver_nostatic', # If using whitenoise

    # Local Apps
    'core',
    'portfolio',
    'blog',
]

```

```
# ... (Middlewares)

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'], # Pointing to global templates folder
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

# Static files (CSS, JavaScript, Images)
STATIC_URL = 'static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
STATIC_ROOT = BASE_DIR / 'staticfiles' # For production

# Media files (User uploads)
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

2. portfolio/models.py

Standard usage of Models with proper string representation and image handling.

```
from django.db import models

class Tag(models.Model):
    name = models.CharField(max_length=50)

    def __str__(self):
        return self.name

class Project(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField()
    image = models.ImageField(upload_to='projects/')
    tags = models.ManyToManyField(Tag)
    github_link = models.URLField(blank=True)
    live_link = models.URLField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['-created_at'] # Newest first

    def __str__(self):
        return self.title
```

3. portfolio/views.py

Use **Class Based Views (CBVs)**. They are cleaner and standard for simple CRUD apps like portfolios.

```
from django.views.generic import ListView, DetailView
from .models import Project

class ProjectListView(ListView):
    model = Project
    template_name = 'portfolio/project_list.html'
    context_object_name = 'projects'

class ProjectDetailView(DetailView):
    model = Project
    template_name = 'portfolio/project_detail.html'
    context_object_name = 'project'
```

4. templates/base.html

This is the "Frontend" logic in Django.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My Portfolio{% endblock %}</title>
    <!-- Example: Using Tailwind via CDN for quick styling -->
    <script src="[https://cdn.tailwindcss.com](https://cdn.tailwindcss.com)"></script>
</head>
<body class="bg-gray-50 text-gray-800">

    <!-- Navbar -->
    <nav class="bg-white shadow mb-8">
        <div class="container mx-auto px-4 py-4 flex justify-between">
            <a href="{% url 'home' %}" class="font-bold text-xl">MyName.</a>
        <div>
            <a href="{% url 'project_list' %}" class="mr-4 hover:text-blue-600">Por
            <a href="{% url 'about' %}" class="hover:text-blue-600">About</a>
        </div>
    </div>
</nav>

    <!-- Main Content Injector -->
    <main class="container mx-auto px-4 min-h-screen">
        {% block content %}
        <!-- Child templates inject content here -->
        {% endblock %}
    </main>

    <!-- Footer -->
    <footer class="bg-gray-800 text-white py-6 mt-12">
        <div class="container mx-auto text-center">
            <p>&copy; 2025 My Name. Built with Django.</p>
```

```

    </div>
</footer>

</body>
</html>

```

5. portfolio/templates/portfolio/project_list.html

How to extend the base and show data.

```

{% extends "base.html" %}

{% block title %}Portfolio | My Projects{% endblock %}

{% block content %}
<h1 class="text-3xl font-bold mb-6">My Projects</h1>

<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
  {% for project in projects %}
    <div class="bg-white rounded-lg shadow overflow-hidden hover:shadow-lg transition">
      {% if project.image %}
        
        <h2 class="text-xl font-semibold mb-2">{{ project.title }}</h2>
        <p class="text-gray-600 mb-4">{{ project.description|truncatewords:20 }}</p>
        <a href="{% url 'project_detail' project.pk %}" class="text-blue-600 font-n
      </div>
    </div>
  {% empty %}
    <p>No projects uploaded yet.</p>
  {% endfor %}
</div>
{% endblock %}

```